

CO5 ASSESSMENT TOOL 2(Pseudocode Writing)

Name: Y.Thanushma

Registration Number: 192425355

Course Code: CSA0617-Design & Analysis of Algorithms

1. Sudoku Solver with Pre-filled Constraints

Input:

- Grid[9][9] – Sudoku grid with pre-filled values.
- FixedDiagonal – fixed values/constraints for the main diagonal.
- Empty cells are represented by 0.

Output:

- A completed Sudoku grid satisfying all row, column, subgrid, and diagonal constraints.

Pseudocode:

ALGORITHM SudokuSolver(Grid)

IF Solve(Grid) = TRUE THEN

 PRINT "Completed Sudoku:"

 PRINT Grid

ELSE

 PRINT "No valid solution exists"

END ALGORITHM

FUNCTION Solve(Grid)

 Find an empty cell (row, col)

 IF no empty cell exists THEN

 RETURN TRUE

```

FOR num ← 1 TO 9 DO

    // Pruning: reject invalid numbers immediately

    IF IsValid(Grid, row, col, num) = TRUE THEN

        Grid[row][col] ← num

        // Recursively solve the remaining cells

        IF Solve(Grid) = TRUE THEN

            RETURN TRUE

        // Backtrack if the choice leads to failure

        Grid[row][col] ← 0

    END IF

END FOR

RETURN FALSE

END FUNCTION

FUNCTION IsValid(Grid, row, col, num)

    // Check row constraint

    FOR j ← 0 TO 8 DO

        IF Grid[row][j] = num THEN

            RETURN FALSE

        END IF

    END FOR

    // Check column constraint

    FOR i ← 0 TO 8 DO

        IF Grid[i][col] = num THEN

            RETURN FALSE

```

```

        END IF

    END FOR

    // Check 3 × 3 subgrid constraint

    startRow ← (row DIV 3) × 3

    startCol ← (col DIV 3) × 3

    FOR i ← startRow TO startRow + 2 DO

        FOR j ← startCol TO startCol + 2 DO

            IF Grid[i][j] = num THEN

                RETURN FALSE

            END IF

        END FOR

    END FOR

    // Check main diagonal constraint

    IF row = col THEN

        IF num violates FixedDiagonal[row] THEN

            RETURN FALSE

        END IF

    END IF

    RETURN TRUE

END FUNCTION

```

Key Points:

- Backtracking tries numbers from 1 to 9.
- Row, column, and 3×3 subgrid constraints are checked before placement.
- Fixed diagonal values are preserved.

- Invalid choices are pruned immediately.
- If a later placement fails, the algorithm backtracks and tries another value.
- The algorithm is complete because it explores every valid possibility until a solution is found.

2. Subset Sum with Negative Numbers

Input:

- Set[0...n-1] – set of positive or negative numbers.
- Target – required sum.

Output:

- All subsets whose sum is equal to Target.

Pseudocode:

ALGORITHM SubsetSum(Set, Target)

Result $\leftarrow$ empty list

CurrentSubset $\leftarrow$ empty list

Backtrack(0, 0)

IF Result is empty THEN

 PRINT "No valid subset exists"

ELSE

 PRINT "Valid subsets:"

 PRINT Result

END IF

END ALGORITHM

```

FUNCTION Backtrack(index, CurrentSum)

    // If target is reached, store the current subset
    IF CurrentSum = Target THEN
        ADD copy of CurrentSubset TO Result
        RETURN
    END IF

    // If all elements are processed, stop
    IF index = n THEN
        RETURN
    END IF

    // Pruning using possible sum bounds
    IF Target is outside the achievable sum range from index THEN
        RETURN
    END IF

    // Include current element
    ADD Set[index] TO CurrentSubset
    Backtrack(index + 1,
        CurrentSum + Set[index])
    REMOVE last element FROM CurrentSubset

    // Exclude current element
    Backtrack(index + 1, CurrentSum)

END FUNCTION

```

Key Points:

- The algorithm considers both include and exclude choices.

- Negative numbers are handled naturally because the current sum can increase or decrease.
- Recursion explores all possible subsets.
- A subset is stored whenever its sum becomes equal to the target.
- Bound-based pruning can stop branches that cannot possibly reach the target.
- Unlike the positive-only case, simple rules such as $\text{CurrentSum} > \text{Target}$ cannot always be used because negative values may later reduce the sum.
- The algorithm generates all valid subsets.

Time Complexity: $O(2^n)$ in the worst case.

Space Complexity: $O(n)$ for recursion and the current subset, excluding the output.

3. Permutation with Restricted Positions

Input:

- $\text{Set}[0\dots n-1]$ – elements to be permuted.
- $\text{Restriction}[\text{element}][\text{position}]$ – indicates whether an element is forbidden at a particular position.

Output:

- All valid permutations satisfying the position restrictions.

Pseudocode:

ALGORITHM RestrictedPermutations(Set, Restriction)

Result $\leftarrow$ empty list

Permutation $\leftarrow$ array of size n

Used[0...n-1] $\leftarrow$ FALSE

Backtrack(0)

IF Result is empty THEN

 PRINT "No valid permutation exists"

```

ELSE

    PRINT "Valid permutations:"

    PRINT Result

END IF

END ALGORITHM

FUNCTION Backtrack(position)

    // Complete permutation generated

    IF position = n THEN

        ADD copy of Permutation TO Result

        RETURN

    END IF

    FOR i ← 0 TO n - 1 DO

        // Pruning: skip already-used elements

        IF Used[i] = TRUE THEN

            CONTINUE

        END IF

        // Pruning: skip restricted positions

        IF Restriction[Set[i]][position] = TRUE THEN

            CONTINUE

        END IF

        // Place the element

        Permutation[position] ← Set[i]

        Used[i] ← TRUE

```

```
// Recursively fill the next position

Backtrack(position + 1)

// Backtrack and remove the element

Used[i] ← FALSE

END FOR

END FUNCTION
```

Key Points:

- The algorithm fills one position at a time.
- A loop tries every unused element for the current position.
- Restricted element-position combinations are rejected immediately.
- The Used array prevents an element from appearing more than once.
- When a complete permutation is formed, it is added to the result.
- Backtracking removes the previous choice and tries another possibility.
- Therefore, all and only valid permutations are generated.

Time Complexity: $O(n \times n!)$ in the worst case.

Space Complexity: $O(n)$ for recursion, permutation, and the Used array, excluding the output.